Adversarial-Mask Attacks: from 2D to 3D Projections

1 Goal

We want to build a small image overlay — a *mask* — that, when laid on top of a picture of a 3D object, makes an image classifier stop recognising what the object is. The same mask has to work from many different camera angles, including angles it never saw while being trained. Think of it as a "filter" you could hold in front of a camera looking at a turtle: the classifier should look at the turtle through the filter and call it anything *except* a turtle.

2 2D pipeline (warm-up)

We first practiced on a single fixed photo. The idea is the same: find a tiny perturbation that we add to the picture so the classifier gets it wrong. The trick is that we want the perturbation to keep working even after the image is rotated, scaled or slightly recoloured — otherwise it would only fool one exact view.

Where the rotation happens. Every step of training works in three pieces:

1. Take the picture and add the current mask on top.
2. Apply a random change to the result: rotate it a bit (up to ±30°), scale, shift, jitter the colours, add noise.
3. Show the changed picture to the classifier and measure how wrong it is.

The order matters. The random rotation sits *between* "image + mask" and "classifier". Because that rotation is built from differentiable operations, the training signal that tells us how to improve the mask travels backwards through the rotation. So each improvement step is computed on a rotated view, not the original. We repeat with many random rotations per step and average, which pushes the mask toward something that works *on average* across rotations — making it robust to angle, instead of overfitting to a single fixed orientation. After the update we keep the mask inside a small brightness range so it stays close to invisible.

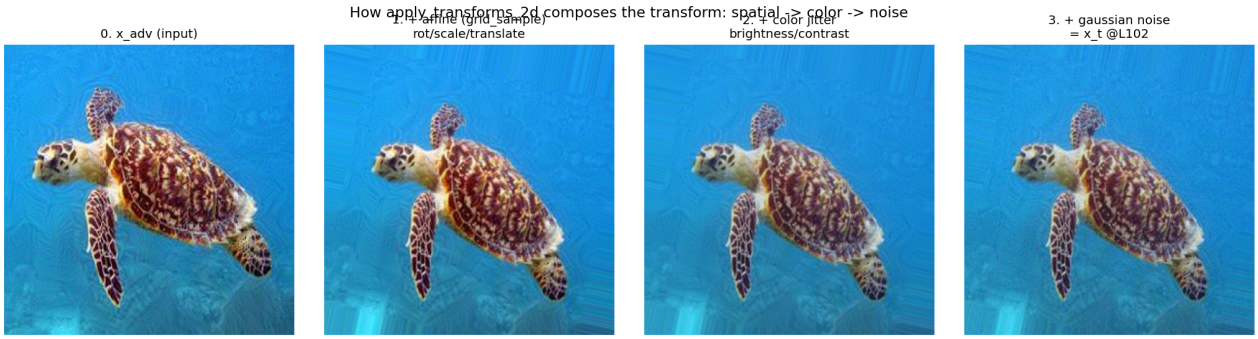


Figure 1: The 2D pipeline. Picture plus mask first, then the random rotation/scale/colour change, then the classifier. The rotation step is exactly where the training signal gets averaged across views, which is what makes the mask work from many angles.

3 3D pipeline

For the 3D case we kept the same overall idea but switched out the source of the random rotation. Instead of rotating the 2D image after the fact, the randomness now comes from the camera that looks at the 3D object. Each training step does this:

1. Pick a small batch of random camera viewpoints around the turtle mesh.
2. Render each viewpoint — this gives us a batch of fresh 2D pictures of the turtle.

3. Lay the current mask on top of every rendered picture.
4. Ask the classifier how likely each result is to be a turtle, and measure the total "turtle-ness" still left.
5. Take one improvement step on the mask that reduces "turtle-ness", then clip the mask back into its allowed range.

Where the rotation happens now. In the 2D pipeline the rotation was a differentiable image transform that the training signal had to travel through. In the 3D pipeline the rotation lives inside the renderer instead. Crucially the mask is added *after* rendering, so as far as the training signal is concerned the rendered picture is just a fixed input — the renderer itself does not need to be differentiable. Fresh viewpoints every step are what make the mask generalise across angles, just like fresh rotations did in 2D.

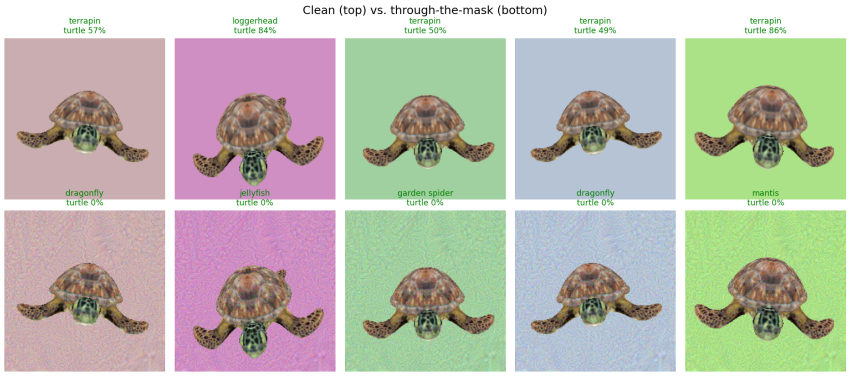
A few engineering details that mattered.

- *Texture.* The simple renderer originally fell back to a flat gray colour, so the clean turtle looked like a gray blob and was unrecognisable. We now read the turtle's texture image from its material file and apply it.
- *Mesh framing.* The raw turtle model is far from the world origin and very large in its own units. We re-centre it and rescale it so camera distance behaves the same way for any input mesh.
- *Viewpoint range.* Picking a completely random orientation produces lots of upside-down or edge-on views that the classifier can't identify in the first place. We restrict to roughly upright, mostly front-facing angles, where the clean turtle reliably reads as a turtle (loggerhead / leatherback / terrapin).
- *Camera "zoom".* A wide-angle camera left the turtle very small in the frame, which made the task too easy — a full-frame mask trivially overwhelms a tiny object. Just moving the camera closer made the perspective unnatural and broke recognition. Switching to a more telephoto setting makes the turtle large in the frame *and* keeps it recognisable — so the mask has to actually fight a confidently recognised turtle.

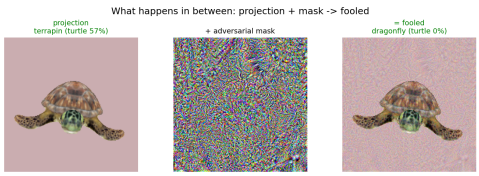
4 Results

We ran the 3D attack on the sea-turtle mesh with 12 training steps and 6 random viewpoints per step. On a laptop CPU it finishes in about three minutes. The mask is trained only on freshly sampled random viewpoints, and the held-out display views in the figure below are never used during training.

- Before the mask: all 5 held-out views are classified as some kind of turtle (loggerhead, leatherback, terrapin), with the classifier assigning 49%–98% of its probability to a turtle class.
- Through the mask: none of the 5 views are turtle any more. The classifier now says dragonfly, jellyfish, garden spider, mantis — turtle probability 0%.
- During training the "turtle-ness" the classifier still sees drops from around 33% to 0% within the first few steps and stays there.



(a) Held-out views. Top row: clean renders, all classified as turtle (green). Bottom row: same renders seen through the mask, none of them turtle any more (green).



(b) One view broken down: clean projection + mask = fooled classifier.

Figure 2: The mask is trained on random camera angles of the whole mesh and then tested on different angles it never saw. It still works, removing the turtle prediction from every test view.

Take-away. The 2D warm-up trained a robust mask by averaging the training signal across random image transforms. The 3D version replaces those transforms with random camera angles from the renderer — with sensible framing, the same trick yields one universal mask that de-classifies a confidently recognised 3D turtle from angles it never saw.